

day-23 Linked List

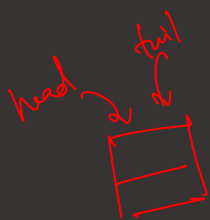
```
class Node {  
    int data;  
    Node next;
```

```
    public Node (int data) {  
        this.data = data;  
        this.next = null;
```

```
    }  
}
```

add first

$O(1)$



new node

head

null

newNode.next = head

head = newNode

head

20

head

tail

null

Print
 $O(n)$

head

10

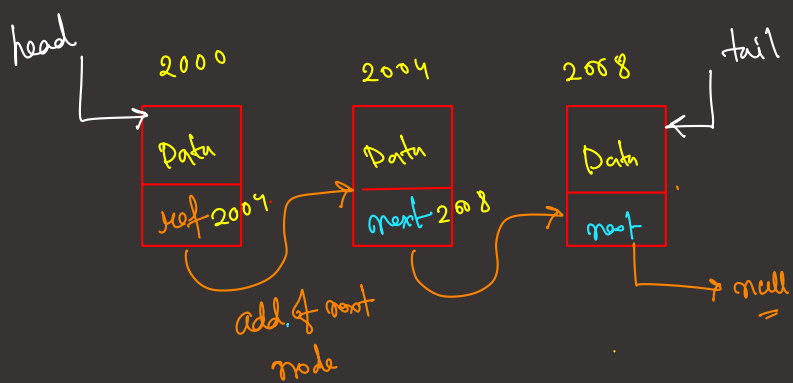
20

30

Cons

null

10, 20, 30



size ++

tail

null

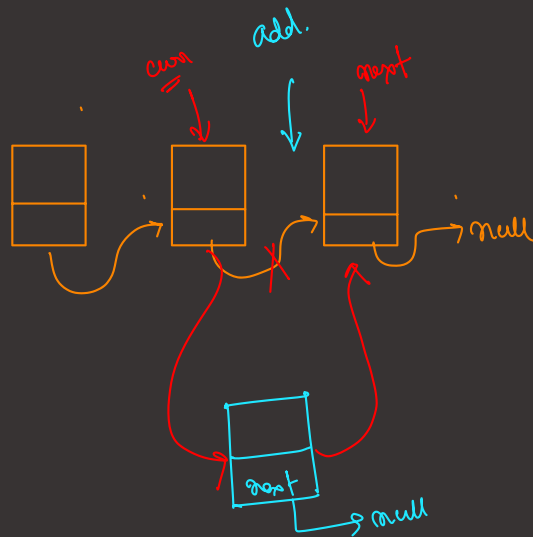
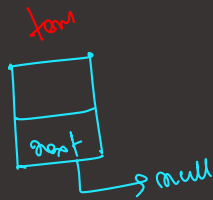
tail

tail

add at any index

idx = 2

$O(n)$



curr, next = temp -
temp, next = curr

20 → 10 → 30

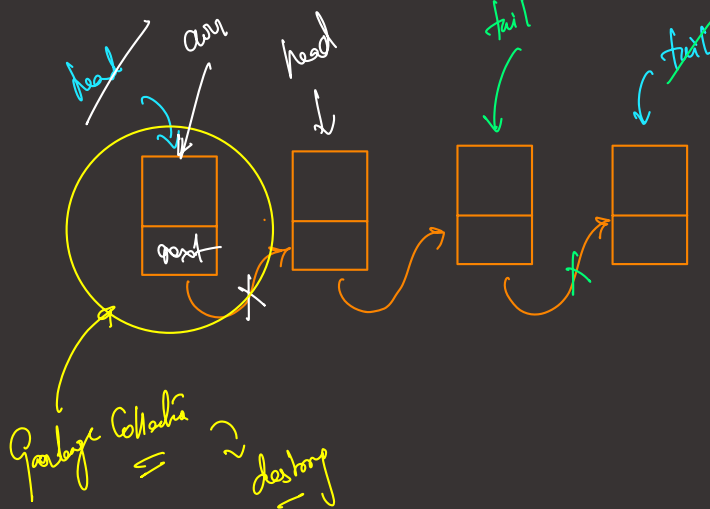
20 → 10 → 50 → 30

initially

curr = head

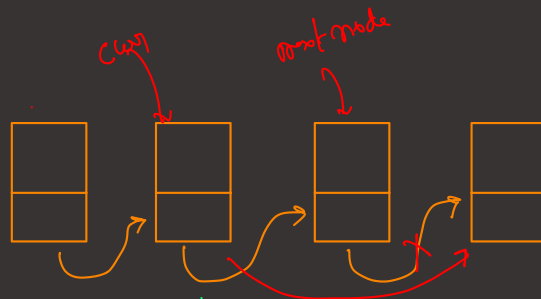
next = head, next

Remove 0 position



Remove any idx

idx = 2



curr = head, nextNode = head, next;

for (int i = 0; i < !idx - 1; i++)

curr = curr, next;

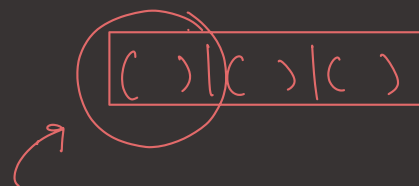
curr, next = nextNode, next;
nextNode = null;

H.W Search in a LL
delete a key.

Priority on for.

Day-20
 class pair {
 int data;
 int freq;
 }

ArrayList < Pair > list = new ArrayList < > ();



Search

↙
while loop

→
Recursion

↖ return index
 Search (head, value, i) {

if (head == null) {
 return -1;

}

if (head.data == value) {
 return i;

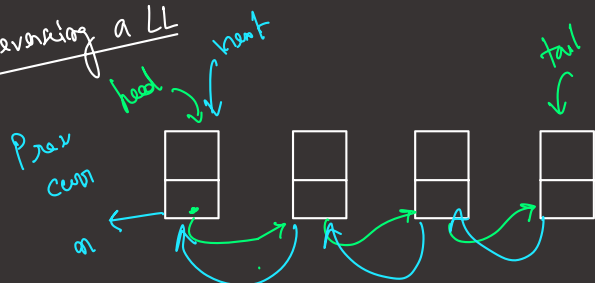
}

return Search (head.next, value, i+1);

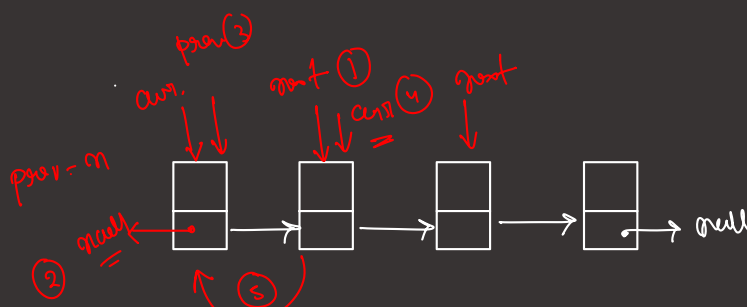
}

Reversing a LL

$\rightarrow O(n)$



prev = null
 curr = head
 next = null
 initial



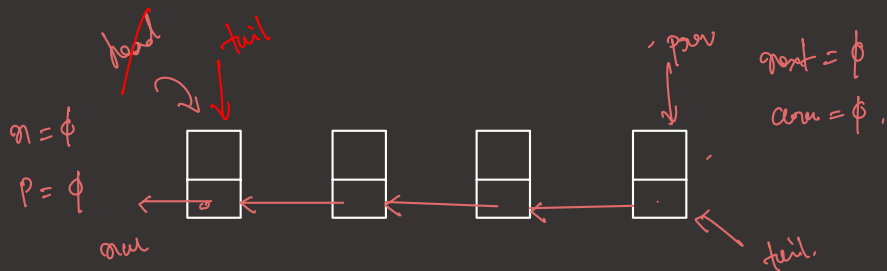
while (curr != null) {
 ① next = curr.next

② curr.next = prev

③ prev = curr

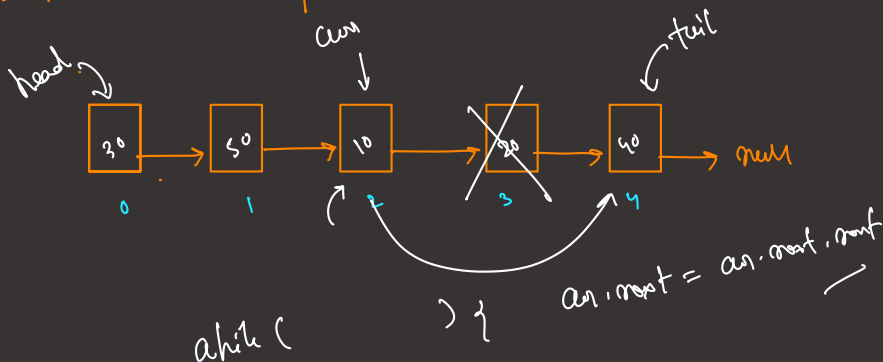
④ curr = next

initial
 prev = null
 curr = head
 next = null



tail = head
 head = prev

find and remove nth node from End

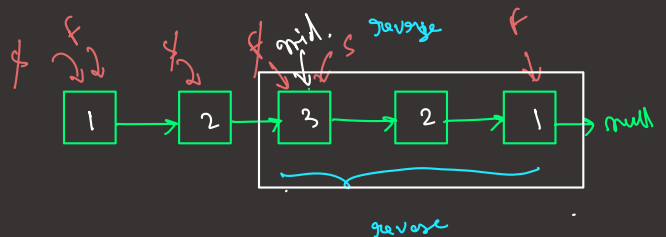
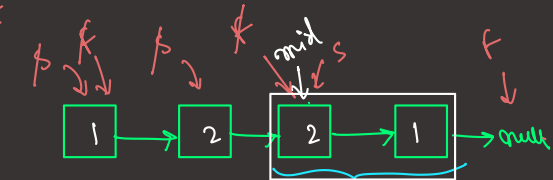


Size = 5
 I have to delete 3rd node
 Size - m
 $5 - 2 = 3$

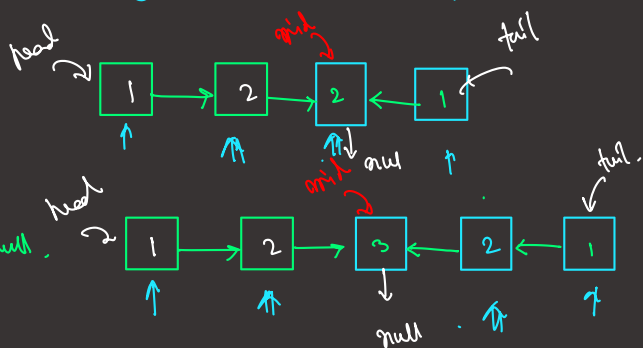
Q Check a linked list is palindrom or not

1 2 2 1 ✓ 1 2 3 2 1 ✓
 2 4 4 2 ✓
 2 4 4 3 ✗

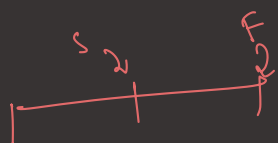
Slow +1
 Fast +2



- ① to find mid node
- ② reverse the 2nd half.



if (f == null or f->next == null)
 Slow → mid

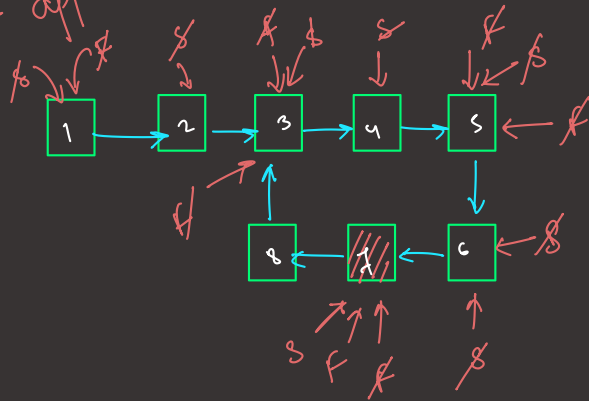


Detect cycle in a Linked List → if there exist a cycle then $s = f$ • Floyd's cycle finding Algorithm.

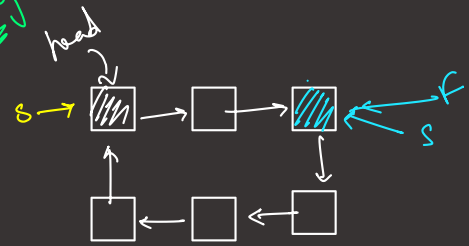
① Slow fast approach

$s+1$

$f+2$



Study



if there exist a cycle then remove it.

① To check there exist a cycle or not $s = f$

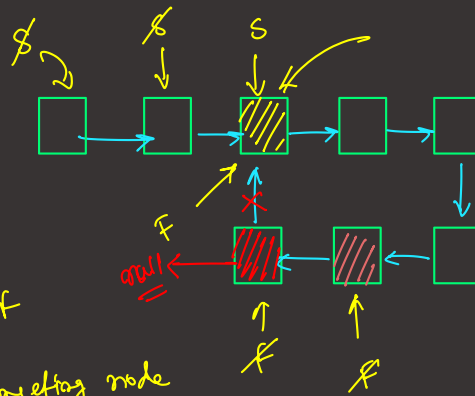
② $s = \text{head}$

$s+1$
 $f+1$

when $s = f$

the meeting node

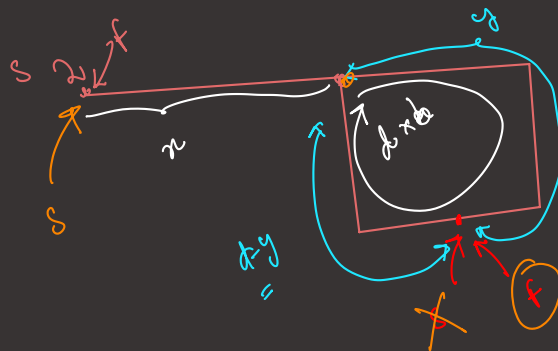
will be point of connecting



③ Prev. track fast previous

$$x = kd - y$$

$$n = (k-1)d + (d-y)$$



$$S_0 = n + ad + y$$

$$F_0 = a + bd + y$$

$$F_0 = 2S_0$$

$$a + bd + y = 2(n + ad + y)$$

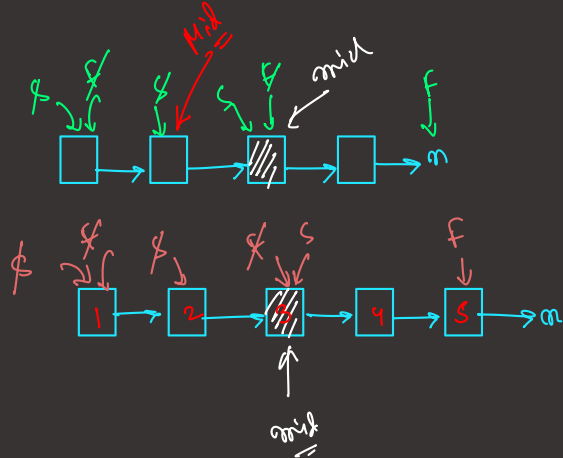
$$\Rightarrow n + y \neq 2ad - bd = 0$$

$$\Rightarrow n = d(b - 2a) - y$$

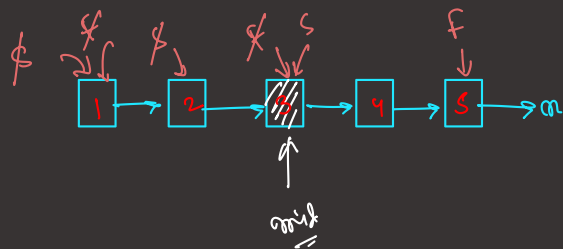
$$n = kd - y$$

Merge Sort

even



odd



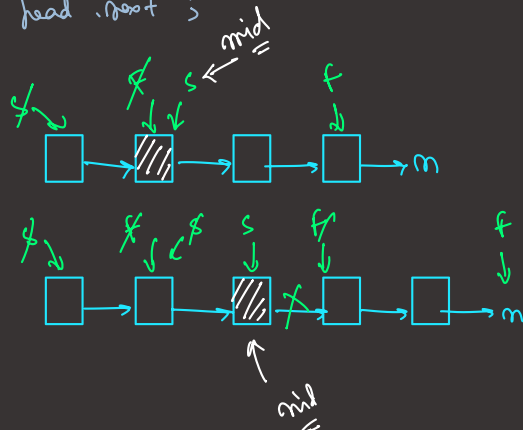
until we reached to single element or null $\rightarrow$ base case =

LH, RH

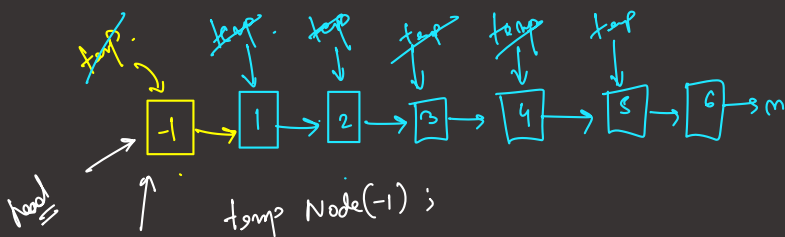
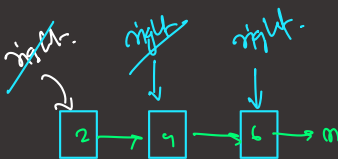
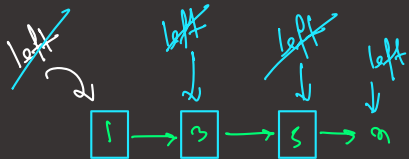
find Mid

slow = head

fast = head.next



mid = return slow



temp Node(-1)

```
while (left != null && right != null) {
    if (left.data < right.data) {
        temp.next = left;
        temp = temp.next;
        left = left.next;
    }
}
```

ms()

1 find mid = find mid()

left = head
right = mid.next
mid.next = null

2 leftH = MS (left half)

rightH = MS (right half)

3 merge (leftH, rightH)

```
else {
```

```
    temp.next = right ;
```

```
    temp = temp.next ;
```

```
    right = right.next ;
```

```
}
```

```
}
```

```
if (left != null)
```

```
    temp.next = left ;
```

```
if (right != null)
```

```
    temp.next = right ;
```

```
head = head.next ;
```

